CBR: Capture, Bind, Recall

Continuation plan for a pilot at Elisa · Since AI 2026, hidden-knowledge challenge · Mikael Salomaa and Abhishek Singh

Repository: github.com/abhishek-xtra/Since-AI-Elisa-Challenge- · Demo: cbr-teams.vercel.app · Video: youtube.com/watch?v=JbLK_Vu-E80

CBR is a context layer that reads the conversations a network team already has (Teams, email), stores each reusable operational rule as a context assertion bound to the network graph (trigger, candidate scopes, action, avoid list, verbatim evidence, trust, validity), and hands it back when the same situation comes up on another unit. It asks the expert one question, once, when the scope is unclear, and says nothing when it knows nothing. It is advisory and never acts on the network.

Where the prototype stands

- **Works end to end on synthetic data.** Elisa's two example conversations plus 18 synthetic scenario folders; nine demo beats from capture to expiry and retraction; 467 tests, an eval over 17 scenarios and 49 adversarial break cases, all runnable offline from cached model calls.
- **Design confirmed by Elisa.** One tap only at reuse, silence over wrong context, an explicit no-prior-knowledge signal, questions owned by a team, secrets kept verbatim for a prototype only. Each answer is reflected in the code or listed below as pilot work.
- **Known limits.** Synthetic topology and threads; exact-key merging (no corroboration across differently worded rules); the JSON store runs, the Neo4j store is written but untested; access details in messages are stored as written; no live connectors.

Pilot: three steps, one team, read-only first

Step	What runs	What we measure and who decides
1. Capture, read-only weeks 1 to 4	One Teams team and one shared mailbox in Autonomous Network Operations, read through Microsoft Graph, read-only. Elisa's topology export replaces the synthetic one (device: model, firmware, install year; site: region, as Elisa suggested). Assertions written to Elisa's graph store behind the existing store interface (Neo4j first, since the store is one interface with two implementations). Nothing is shown to engineers yet.	Tips captured per week; share of extracted excerpts that pass the verbatim check; a weekly sample of captured rules reviewed by the team's experts: correct trigger, correct action, correct avoid list, sensible candidate scopes. The team lead decides what counts as a good capture.
2. Recall for people weeks 5 to 8	When an incident thread starts, the layer posts its result in that thread: a proposal with source, scope and date, a decline with its reason, or one question to the owning team. Outcomes are recorded from the thread ("did that, traffic is back") and re-derive trust. Expiry, supersession and retraction run as in the prototype.	Proposals the on-call engineer followed, declined, or corrected; questions asked per tip and whether any was asked twice; time from a tip's capture to its first reuse on another unit; expert interruptions per week compared with the team's own count today. Engineers can mute it per thread; the team lead decides whether step 3 starts.
3. Recall for the agent weeks 9 to 12	`get_context` exposed as a tool to Elisa's network operations agent, with the `prior_knowledge` signal so the agent knows when it is on its standard procedure. The agent's own escalation thread becomes a capture channel, as Elisa described in the briefing. Still advisory: the agent proposes, a human acts.	Incidents where the agent had prior knowledge versus none; proposals the human accepted; declines whose reason the human agreed with. Elisa decides whether and where the agent may act on a sourced, verified tip; that confidence model is Elisa's call, not ours.

Security layer: two gates, built in step 1

Today the prototype stores what people wrote, verbatim, and shows a proposal to whoever asks. A pilot on real channels needs two gates around that, both deterministic code, both logged.

- **Capture gate: what never enters the store.** Credentials, tokens and passwords are detected by pattern and dropped before extraction; a rule that depends on one is stored with a pointer to the source message instead of the secret. Personal

remarks, HR or customer matters and anything outside the operational vocabulary produce no assertion at all; the extractor already returns nothing for chit-chat, and the pilot adds a blocklist of topics the team names. An engineer can say "don't write this down" in the thread, and that message and its replies are skipped, the same way a retraction is honoured today. Access details that are the fact itself (where a key is kept, a gate code) are stored, but flagged *sensitive* at capture and held in a review queue until the team lead clears them.

- **Recall gate: who may see what.** Every proposal is released to a person and a context, not broadcast. A sensitive detail is shown only to the engineer dispatched to that site, in that incident's thread, for the duration of the dispatch; everyone else, including the autonomous agent, sees that a tip exists and who to ask. Roles come from Elisa's directory, so the layer never keeps its own user list. Each disclosure is written to an audit log with who, what, when and the incident, and the log is the first thing the weekly review reads.
- **Data minimisation.** Verbatim excerpts are kept because they are the evidence a judge or an engineer checks; they are limited to the sentences the rule rests on, carry the retention period Elisa sets, and are deleted with the assertion when it is retracted past review. Nothing is sent outside Elisa's cloud once the model runs there.

Engineering work the pilot needs

- **Connectors.** Microsoft Graph for Teams and mail; the existing adapters already produce the message contract, so this is one adapter per channel. Phone calls later, via transcripts.
- **Store.** Finish and test the Neo4j store against the same tests the JSON store passes; the graph export already exists.
- **Team routing.** Questions go to the team that owns the domain; anyone on it can answer; until then the rule stays usable but marked ambiguous in scope. The question already carries a team field; the delivery into Teams is new.
- **Corroboration.** Merge rules that mean the same thing under different keys, with an expert confirming the first time two keys are joined. The hook exists; the rule does not.
- **Model.** The model sits behind one function that takes a prompt and a schema. Elisa's preferred model, in Elisa's cloud, replaces Gemini by changing that function. Every call is cached and capped.
- **Data protection.** Processing agreement, retention period for verbatim excerpts, and the team's consent before its channels are read. Only the team's own channels, never customer data.

What does not change

The frozen contracts (message, topology, assertion, vocabulary, predicates, get_context). The rule that the model reads and the code decides: scope, trust, outcome, supersession, expiry and the question are deterministic and tested without a model. No numeric confidence. Verbatim evidence or nothing. One question, once. Silence when nothing is captured.

Risks, and what we do about them

- **Real threads are messier than the examples.** The adversarial break tests were aimed at this (typos, chit-chat, hunches, conflicting advice, a fix that failed). Step 1 is read-only so misreads cost nothing; the weekly expert review is the correction loop.
- **Wrong scope.** The layer never guesses a scope: it keeps every candidate and narrows by cases and one answer. A wrong proposal is declined by the engineer in the thread and recorded as a failed outcome, which lowers trust and can send the rule to review.
- **Expert fatigue.** One question per distinct ambiguity, routed to the team. If the count of questions per week rises above what the team accepts, questions are batched or stopped and the rules stay ambiguous.
- **Trust in the agent path.** Step 3 starts only after step 2 shows engineers following proposals. Acting on the network is out of scope for the pilot.

What we need from Elisa

One team willing to be read; one topology export with the attributes above; a graph store endpoint or permission to run Neo4j; a sponsor in Autonomous Network Operations who reviews the weekly sample; and the answers to two open questions from the briefing: whether a rule's validity can end on a condition such as a firmware upgrade rather than a date, and who may close an open edge when the original expert has left.